Upasthiti – Smart Attendance: Project Blueprint & Claude Code Kit

Oct 8, 2026 · @Tushar Gawande

1. Project overview

We are building **Upasthiti – Smart Attendance (project** name), a self-hosted, multi-plant attendance platform for Sheet Shapers that marks attendance only when the employee is physically present, verified by face (permanent staff) or a secure QR code (temporary/hourly workers), and shows the admin the salary payable to each employee based on attendance.

Client: Sheet Shapers (sheetshapers.com), a manufacturing organisation with several plants at different locations. The project is a sponsored final-year engineering project built by a team of 4.

The problem today. Employees mark "proxy" attendance through the existing app or website while they are not on the premises. Temporary workers change week to week, so enrolling their faces is slow and impractical. The company also wants no licensing fees in future and needs the system to run for years with little or no maintenance.

What the client asked for

- QR-based attendance with a unique QR per employee, mainly for temporary, hourly workers.
- Face-recognition attendance for permanent employees.
- Geo-fencing so attendance is accepted only from inside a plant's boundary.
- Support for multiple plants and both employee types.
- Works with hardware the company will provide (tablets, cameras, scanners).
- No paid third-party services or licences; compliant with Indian government rules.
- Payable salary estimate per employee from attendance and a fixed salary structure (added after the phone call with the client); full payroll is a later phase.

Our goals (how we beat the brief)

1. **Zero proxy attendance.** Combine several independent proofs (place, person, device, time) so faking one is not enough.
2. **Fully automated.** Shifts, late marks, overtime, hours for hourly workers and reports are computed with no manual entry.

3. **Easy to run after handover.** Simple setup and few moving parts, so the company can keep it running without a dedicated software team (we are students and cannot offer long-term maintenance).
4. **Payroll as future scope.** Only the payable-salary estimate is in scope (see 2.5). Full payroll, statutory deductions, reports and filings are future scope.
5. **A product that looks and feels premium.** Modern, animated, bilingual UI that impresses examiners and is easy for shop-floor workers.

2. Features: what we will build

The core five modules satisfy the brief; the "value-add" modules are what make this a standout project. Phase 1 is the must-have demo; Phase 2 is the polish; Phase 3 adds automation, reports and the payable salary estimate.

2.1 One attendance channel: the gate kiosk (final decision)

Decision (team): Upasthiti is **one website with one attendance channel: the gate kiosk.** There is no employee phone app for now; a mobile PWA can be added later if the client needs it (for example for field staff). Both permanent and temporary employees, and both face and QR attendance, use the same kiosk screen at the plant gate.

Part of the website	Who uses it	What it does
Kiosk screen (full-screen mode on a gate device)	All employees, at the plant gate	Permanent staff look at the camera (face + liveness). Temporary workers show their QR ID card to the same camera. Check-in and check-out both happen here.
Admin dashboard	Super admin, plant admin, HR, supervisors	Setup, employees, QR cards, face enrolment, live headcount, reports, corrections, and later the salary estimate.

Why kiosk only: the gate device belongs to the company, is registered to one plant and stays at the gate, so attendance from home is impossible by design. A phone app would need many extra checks (GPS spoofing, device binding, per-employee geofence) and still be weaker. One channel also means less code, less testing and fewer things to maintain.

Gate device: any tablet, Android phone or laptop with a camera and a browser. No special attendance machine is needed. During development, team laptops and phones act as kiosks.

Geo-fencing still applies: each kiosk device sends its own GPS position (and network address) with every punch, and the server rejects punches if the kiosk is outside its plant's geofence. This stops someone from taking a kiosk tablet away from the gate.

2.2 Core modules (Phase 1)

1. **Organisation setup:** plants, departments, shifts, geofence drawn on a map (circle or polygon), registered kiosk devices per plant.
2. **Employee management:** permanent and temporary profiles, contractor/agency link for temporary workers, bulk import from Excel, photo, ID card print with QR.
3. **Face attendance:** one-time enrolment with consent, face detection + recognition, **liveness check** (blink or head-turn) to defeat photos and videos.
4. **QR attendance:** unique signed QR per worker, printable ID card, validity dates, instant revoke when a worker leaves.
5. **Geo-fencing & anti-proxy engine:** the server re-checks every punch: registered kiosk device, the kiosk's own GPS position inside the plant geofence, plant network, face/QR validity and time rules (see 2.4).
6. **Attendance processing:** check-in/out pairing, shift matching, late/early/half-day, overtime, total hours for hourly workers.
7. **Admin & HR dashboard:** live headcount per plant, today's present/absent/late, trends, exceptions to review.
8. **Reports & exports:** daily muster, monthly register, hours per temporary worker per contractor, Excel and PDF.
9. **Roles & security:** Super Admin, Plant Admin, HR, Supervisor, plus a registered identity for each kiosk device; full audit log of every change.
10. **Payable salary estimate:** fixed salary structure per employee (Basic, DA, HRA, Travel and other allowances) combined with attendance to show the admin the salary payable each month, with Excel export (see 2.5). Built last, after all other features.

2.3 Optional value-add features (build only if time remains; NOT promised to the client)

- **Emergency muster roll:** one tap shows exactly who is inside each plant right now, for fire drills and evacuations.
- **Offline-first kiosk:** if the network drops, punches are stored on the device, signed, and synced later. No lost attendance.

- **Kiosk health check:** admin sees which gate devices are online, their last punch time and battery level, and gets an alert if a kiosk goes offline.
- **Regularisation workflow:** employee requests a correction (forgot to punch), supervisor approves, everything audited.
- **Leave & holiday calendar:** per plant, feeds attendance status automatically.
- **Anomaly alerts:** impossible travel between plants, repeated failed face matches, punches at odd hours, buddy-punching patterns.
- **Contractor portal:** agencies see hours of their workers; useful for contract-labour compliance and billing.
- **Safety induction gate:** a temporary worker's QR activates only after the safety induction is marked complete.
- **Multilingual UI:** English, Marathi and Hindi, with large touch targets and voice/beep feedback on the kiosk.
- **Notifications:** email via the company's own mail server; in-app alerts. No paid SMS.
- **Analytics:** attendance %, overtime cost trends, absenteeism heat-map, plant comparison.

2.4 Anti-proxy checks on every punch

The server, not the kiosk, decides. A punch is accepted only if every check passes; otherwise it is rejected or flagged for HR review, with the reason stored.

Check (all at the kiosk)	What it stops
Registered kiosk device (signed device token)	Attendance from any device other than an approved gate kiosk
Kiosk's own GPS inside the plant geofence, plus plant network / IP allow-list	A kiosk tablet taken outside the plant
Face match above threshold + liveness (blink / head turn)	Using someone's photo or video (permanent staff)
QR signature valid, not expired, not revoked	Fake, copied or old QR cards (temporary workers)
Photo snapshot taken at every QR punch, kept for a short period for HR review	One worker showing a friend's QR card (buddy punching)
Server time only + duplicate-punch cool-down	Changed device clock, double punches
Impossible-travel check across plants	The same QR card used at two plants too quickly

2.5 Payable salary estimate (in scope, built last) and full payroll (future scope)

Added after the phone call with Sheet Shapers: the admin must be able to see how much salary is payable to each employee based on his attendance. Sheet Shapers has a **fixed salary structure** for its employees.

How it is calculated (per employee, per month)

1. **Monthly salary** = Basic + Dearness Allowance + HRA + Travel Allowance + other allowances (saved once by HR; components are configurable).
2. **Per-day salary** = monthly salary divided by salary days (26, 30 or actual days of the month; a setting, to be confirmed by the client).
3. **Payable days** = days present + paid leave + paid holidays / weekly offs (rules to be confirmed by the client).
4. **Earned salary** = per-day salary x payable days.
5. **Overtime pay** = overtime hours x overtime rate (rule to be confirmed by the client).
6. **Deductions** (PF, ESIC, TDS, advances) are typed in by the admin. The system does not calculate them.
7. **Net payable** = earned salary + overtime pay - deductions.

Temporary / hourly workers: total hours worked x hourly rate (plus overtime if applicable).

Example (illustrative): monthly salary Rs. 30,000 with 30 salary days gives Rs. 1,000 per day. 18 payable days gives Rs. 18,000. 4 overtime hours at Rs. 150 adds Rs. 600. The admin enters deductions of Rs. 2,000. Net payable shown: Rs. 16,600.

Output: a monthly salary sheet on screen and as an Excel file with, for every employee, days present, absent, leave, overtime hours, payable days, earned salary, overtime pay, deductions and net payable. Screens are labelled "Estimate".

Rules for Claude Code

- Use exact decimal arithmetic for money, never floating point.
- Keep the calculation in one small, well-tested module; salary-day basis, overtime rule and paid-day rules are settings, not hard-coded.
- Store salary structures with an effective-from date so old months are not changed when a structure changes.
- Do not build anything from the future-scope list below.

Future scope (not promised, not to be built now): automatic PF / ESIC / Professional Tax / TDS calculation, statutory reports (such as the PF ECR file), challan generation, Form 16, tax filing, payslips, employee document storage, compliance due-date reminders, and alerts about changes in government rules. Reasons: legal risk, government portal access, yearly rule updates, and limited final-year time.

Open client questions (Q9 and Q10 in the proposal PDF): salary components, salary-day basis, whether paid weekly offs / holidays / leave count as payable days, overtime rate, hourly-rate rule for temporary workers, and a sample salary structure for testing.

3. Government compliance (India)

Face data is personal data, and attendance records are statutory records, so compliance shapes the design from day one. The table maps each law to concrete features. This is engineering guidance, not legal advice: confirm the final list with Sheet Shapers' HR and legal team, because labour-code rules and state rules (Maharashtra) are still being rolled out.

Law / rule	What it requires (in short)	What we build
Digital Personal Data Protection Act 2023 + DPDP Rules 2025	Clear notice, purpose limitation, data minimisation, security safeguards, breach reporting, right to access/correct/erase, retention limits	Consent screen at face enrolment (English/Marathi/Hindi), privacy notice page, store only face **embeddings** (not photos) encrypted at rest, retention policy with auto-purge, data-request workflow, breach log
IT Act 2000, Sec. 43A + SPDI Rules 2011 (biometrics = sensitive data)	Reasonable security practices for biometric data	AES-256 encryption at rest, TLS in transit, role-based access, audit trail, security policy document
Labour Codes (Code on Wages, OSH & Working Conditions Code, Social Security Code, Industrial Relations Code)	Working-hour limits, overtime rules, weekly rest, electronic registers, contract-labour records	Configurable daily/weekly hour limits and overtime rules per plant, digital muster roll and register exports, contractor-wise worker records
Factories Act 1948 / Maharashtra Factories Rules (where still applicable)	Muster roll, adult worker register, hours of work	Register formats exportable as PDF/Excel
Maharashtra Shops & Establishments Act 2017 (offices)	Attendance and working-hour records for office staff	Same engine, office plants configured separately
Minimum wages / floor wage	Hourly workers paid at least the notified rate	Hourly rate field with minimum-rate validation (used in the salary estimate)
Record retention	Keep statutory records for the required years	Immutable attendance table, soft deletes only, configurable retention
Accessibility & fairness	Equal treatment, alternative method if face fails	Fallback: supervisor-verified QR/manual punch, fully logged

Key privacy-by-design rules: collect the minimum, never store raw face images after enrolment, never send biometric data to any outside service, let an employee opt for QR instead of face, and delete biometric data when an employee leaves.

4. Tech stack: Claude Code will analyse and recommend

We are not fixing the tech stack in advance. In the planning session (section 7), Claude Code studies the full project and recommends a **modern, current** stack, explaining each choice. The only hard rule is cost: nothing that the team or Sheet Shapers must pay for, now or later.

Selection criteria Claude Code must follow

1. **Zero cost:** free and open-source (MIT, Apache-2.0, BSD or similar). No paid APIs, per-use fees, subscriptions or commercial licences.
2. **Modern and well supported:** current, actively developed tools with good documentation and community, not outdated or legacy frameworks.
3. **Fits the problem:** strong camera, face recognition and QR support in the browser and/or server; works on low-cost tablets and phones; offline support for the gate screen.
4. **Impressive UI:** supports rich animations and a premium design system.
5. **Privacy:** face data processed inside our own system, never sent to an outside AI service.
6. **Easy to run:** simple deployment the company can keep running without us.

Things to avoid (they charge money or have restrictive licences): paid face APIs (AWS Rekognition, Azure Face), Google Maps paid API, paid SMS (Twilio), paid auth (Auth0, Clerk paid tiers), component kits or animation libraries with commercial licences, and InsightFace's pre-trained models (non-commercial only).

Hosting options to compare (Claude Code should recommend one)

Decision (team): deploy on free tiers first, Docker on the company server later.

Stage	Where	Notes
Development + college demo (now)	Frontend on **Vercel** (free Hobby plan); backend and database on a free tier such as **Render** or **Railway**, or another free Postgres host	Gives free HTTPS, which the browser needs for camera access. Free tiers have limits: Vercel Hobby is for non-commercial use, Render free services sleep when idle (first punch of the morning can be slow), Railway gives limited trial credit. Claude Code must check today's limits before choosing.

Stage	Where	Notes
Final delivery to Sheet Shapers (later)	**Docker** package on the company's own computer/server	No monthly bills, and employee face data stays inside the company. Decide after the free-tier version works.

Rule for Claude Code: keep the app portable (configuration through environment variables, no vendor-only features), so moving from free tiers to Docker later is easy. Deploy a skeleton early (M1) and keep deploying after every milestone.

5. Team plan

Not planned yet; the team will decide work distribution later. Constraint to remember: 4 members, 2 Claude Code seats, and all members should contribute through the shared GitHub repository.

6. Specification-driven development flow

We build **milestone by milestone** (M0 to M6). Every milestone passes through the same six gates; Claude never writes production code for a milestone before a human has approved both its spec and its plan.

1. **Specify.** Ask the `spec-writer` subagent: `Write the spec for milestone M<n> using docs/specs/_TEMPLATE.md`. Output: `docs/specs/M<n>-<name>.md` covering every feature in that milestone: problem, user stories, API contract, data model, edge cases, acceptance criteria.
2. **Review the spec.** The team reads it, comments, answers Claude's open questions, marks it `Approved`.
3. **Plan.** Switch to **Plan Mode** (Shift+Tab twice): `Read docs/specs/M<n>-<name>.md and produce an implementation plan`. Save it as `docs/plans/M<n>-<name>-plan.md`: tasks in order, files, DB changes, tests, risks.
4. **Review the plan.** Check it against `CLAUDE.md` rules. Approve.
5. **Build.** Exit Plan Mode. Claude implements the plan task by task on branch `milestone/M<n>-<name>`, with tests and small commits.
6. **Validate.** Run the test, quality and security reviewer subagents; you test manually against the acceptance criteria; open a PR; a teammate approves; merge; mark the milestone `Done`.

Golden rules: one milestone per branch; `/clear` between big tasks so Claude's context stays clean; specs and plans live in the repo; if Claude drifts, press Esc and point it back to the spec.

7. Starter kit and kickoff prompt (copy-paste ready)

Use the **Upasthiti starter kit** (zip file): it already contains `CLAUDE.md`, this kickoff prompt as `KICKOFF_PROMPT.md`, the 8 subagents in `.claude/agents/`, the MCP config `.mcp.json`, and both PDFs (proposal and this blueprint) in `docs/client/`. Unzip it into an empty folder, open Claude Code in that folder in normal mode (not Plan Mode), and paste the prompt below. Claude Code does the setup itself (GitHub repo, plugins, MCP servers, design research), then asks you to switch to Plan Mode for planning. You only approve its commands and do one-time logins.

```
You are the lead architect and senior engineer for "Upasthiti – Smart
Attendance", a multi-plant attendance system for Sheet Shapers (a
manufacturing
company in India). This is our final-year engineering sponsored project.

FIRST read, fully: CLAUDE.md, then
docs/client/Sheet_Shapers_Smart_Attendance_Proposal.pdf
and docs/client/Upasthiti_Project_Blueprint.pdf. CLAUDE.md wins if they
differ.

This session has two parts. Do PART A now (normal mode). Do NOT write any
application code in either part. Ask before anything that costs money, makes
the repo public, or installs from an unknown source.

==================== PART A: SET UP THE PROJECT (normal mode)
====================

A1. TOOLS CHECK. Check git, gh (GitHub CLI), node (LTS), npm/npx and python
3.12.
    Detect my OS. For anything missing, give me the exact install commands
and
    wait. Run `gh auth status`; if I am not logged in, ask me to run
    `gh auth login` myself and wait.

A2. GITHUB REPO. Initialise git here (branch main), make a first commit
    "chore: project setup from starter kit", then create a PRIVATE repo with
    `gh repo create upasthiti --private --source=. --push`. (If that name is
    taken in my account, suggest upasthiti-sheetshapers.) Try to protect main
    (require a pull request + 1 approval). If GitHub says my plan does not
    allow it, tell me (I may get GitHub Pro via the Student Developer Pack);
```

```
    do NOT make the repo public. Create labels: spec, plan, build, review,
    bug. Create a GitHub Project board with columns Spec, Plan, Build,
    Review, Done if gh allows it.

A3. PLUGINS. From the official Anthropic marketplaces only, install:
    feature-dev, security-guidance, pr-review-toolkit (or code-review),
    commit-commands, frontend-design; and from Anthropic's skills
    marketplace: webapp-testing and the document skills (xlsx, pdf).
    First list the marketplaces to confirm the exact current names. If you
    can install them through the `claude plugin` CLI, do it; otherwise give
    me the exact `/plugin ...` commands to paste, in order, and wait.

A4. MCP SERVERS. .mcp.json already defines playwright and context7. Verify
    they work (`claude mcp list`). On native Windows, if an npx server
    fails, rewrite it to use "cmd" with args ["/c","npx",...]. Do NOT add
    GitHub MCP (use the gh CLI instead). Ask me before adding Figma MCP
    (only if I share Figma files) or 21st.dev Magic MCP (needs an API key).

A5. SUBAGENTS. Eight subagents already exist in .claude/agents/. Read them,
    check they match CLAUDE.md, fix anything wrong, and tell me what each
    does in one line. Do not add new ones without asking.

A6. DESIGN RESEARCH. Use the ui-designer and feature-researcher subagents
    (WebSearch/WebFetch) to study public examples of attendance/HR
    dashboards, kiosk / check-in screens and admin panels on Godly,
    Dribbble, Awwwards, Mobbin, Figma Community, 21st.dev, Aceternity UI
    and Magic UI. Also look at any screenshots I put in docs/design-refs/.
    Write docs/design-refs/inspiration.md with: links, what to borrow from
    each, and 2 design directions (colours, fonts, motion style, kiosk
    screen idea) for me to choose from. Only free, MIT-type components.
    Remember: kiosk users include shop-floor workers, so the kiosk must be
    big, simple, bilingual-ready and readable in daylight.

A7. Commit and push everything ("chore: tooling, agents and design
research").
    Then give me a short checklist of what is done, what I must still do,
    and say: "Part A done. Switch to Plan Mode (Shift+Tab twice) and type:
    Start Part B". Then STOP.

==================== PART B: PLAN THE PROJECT (Plan Mode)
====================

B1. UNDERSTAND. Restate in your own words: the problem; the single kiosk
    channel (no phone app for now; a mobile PWA may come later if needed);
permanent = face + liveness, temporary = QR +
    snapshot; the anti-proxy checks; and the salary estimate (built last,
```

```
    deductions typed by admin, no statutory calculation).

B2. TECH STACK. Analyse the whole project and recommend the best MODERN,
    current stack. Not outdated/legacy tools. Hard rule: ZERO COST - free
    and open-source (MIT/Apache-2.0/BSD or similar), no paid APIs, usage
    fees, subscriptions or commercial licences, now or after handover.
    Cover: frontend (kiosk screen + admin dashboard), UI components +
    animation, backend, database, face detection/recognition + liveness
    (runs inside our system, CPU is fine, licence allows commercial use),
    QR generate/scan, maps + geofence editor, auth, offline kiosk queue,
    Excel/PDF reports, testing, CI. For each layer: pick, why it fits THIS
    project, licence, and 1-2 rejected alternatives with reasons.

B3. HOSTING. Team decision: free tiers first (frontend on Vercel; backend +
    database on Render, Railway or another free host), Docker on the
    company server later. Check today's free-tier limits and recommend the
    exact combination. Keep everything portable.

B4. ARCHITECTURE. Components, data flow for a face punch and a QR punch at
    the kiosk, kiosk registration, offline sync, and the folder structure.

B5. DATA MODEL. Tables + relationships: plants, geofences, kiosk devices,
    employees, contractors, shifts, face templates, consents, QR
    credentials, punches (+ snapshots), attendance days, regularisations,
    audit logs, salary structures (components, effective dates), monthly
    salary estimates, manual deductions.

B6. MILESTONES. Use the roadmap in CLAUDE.md (M0-M7). For each: features,
    exit criteria, and what is needed for the first demo. Salary estimate
    is last (M7). Do not add features outside scope.

B7. QUESTIONS. Ask me everything you would otherwise guess (plants, gate
    devices, shifts, worker counts, languages, snapshot retention, salary
    rules). Group them; max 15.

After I approve the plan: create docs/SRS.md (outline in the blueprint),
docs/specs/_TEMPLATE.md, docs/architecture.md,
docs/decisions/0001-tech-stack.md, fill section 4 and 9 of CLAUDE.md, commit,
push, and tell me to start M0 with the spec-writer subagent.
```

8. CLAUDE.md (save at the repository root)

Copy the block below into `CLAUDE.md`. Keep it under about 300 lines; update the Roadmap checkboxes as milestones finish.

```
# CLAUDE.md – Upasthiti Project Context

## 0. Source Documents (read first)
- docs/client/Sheet_Shapers_Smart_Attendance_Proposal.pdf: what the client
approved.
- docs/client/Upasthiti_Project_Blueprint.pdf: full project blueprint.
- If the proposal and this file differ, THIS FILE and the blueprint win: they
hold
  the team's latest decisions (e.g. kiosk-only, no employee phone app).

## 1. Project Context / Overview
- Product: Upasthiti – Smart Attendance. Client: Sheet Shapers
(manufacturing, India).
  Final-year engineering sponsored project, team of 4, lead: Tushar Gawande.
- Problem: employees mark proxy attendance from outside the premises;
temporary
  hourly workers change often, so face enrolment is impractical for them.
- Solution: ONE website, ONE attendance channel = the gate KIOSK.
  - Kiosk: a tablet/phone/laptop with camera fixed at each plant gate,
running
    our kiosk screen in full-screen. All employees punch here (check-in/out).
  - Permanent employees -> face recognition + liveness (blink / head turn).
  - Temporary / hourly workers -> unique signed QR code on an ID card, shown
to
    the kiosk camera. A photo snapshot is saved at each QR punch for HR
review.
  - No employee phone app for now: all punches happen at the kiosk.
    A mobile PWA may be added later, only if needed.
  - Admin dashboard (same website): setup, employees, QR cards, face
enrolment,
    live headcount, reports, corrections, salary estimate (built last).
  - The server re-checks every punch (anti-proxy engine). Never trust the
client.
- Roles: SUPER_ADMIN, PLANT_ADMIN, HR, SUPERVISOR (+ KIOSK device identity).
- Must: zero cost (no paid APIs/licences), Indian compliance (DPDP Act 2023,
  IT Act 43A, Labour Codes), simple to run after handover (students cannot
  provide long-term maintenance).
- Scope = client-approved proposal v1.1 + the decisions in this file.
  Future scope (NOT now): full payroll, automatic PF/ESIC/PT/TDS calculation,
  statutory reports, challans, Form 16, tax filing, payslips, employee
document
  storage, compliance due-date reminders, regulatory-change alerts,
  mobile PWA for phone punching (only if needed).

## 2. Architecture & Project Structure
- To be finalised in the planning session. Claude Code recommends the
```

```
  architecture and folder structure; after approval, record it here and in
  docs/architecture.md.
- Fixed folders: docs/ (client/, specs/, plans/, decisions/, design-refs/),
  .claude/agents/ (subagents), .claude/commands/ (custom commands).
- Modules: org setup (plants, geofences, kiosk devices, shifts), employees,
  enrolment (face, QR), kiosk punch, anti-proxy engine, attendance
processing,
  dashboards, reports, audit, auth, salary estimate (last).

## 3. Coding Style
- Readability first: code must be easy to read and understand for a student
  seeing it for the first time.
- Clear, descriptive names for variables, functions and files. No cryptic
  abbreviations.
- Small functions that do one thing. Avoid clever one-liners.
- Add comments where needed: explain WHY and any non-obvious logic
  (anti-proxy rules, attendance and salary calculations, face matching).
  Do not comment obvious lines.
- Each file/module starts with a short comment saying what it is for.
- Consistent formatting using the project's formatter and linter.
- Conventional Commits: feat:, fix:, docs:, test:, refactor:, chore:.

## 4. Preferred Libraries & Tools
- TO BE FILLED after the planning session with the approved stack
  (see docs/decisions/0001-tech-stack.md).
- Rule: modern, actively maintained, free and open-source only.

## 5. Critical Rules (What to Avoid)
- NEVER add anything that costs money: paid APIs, paid face/AI services,
  paid maps, paid SMS, paid auth, commercial-licence UI kits or animation
  libraries, InsightFace pretrained models. If unsure about a licence, ASK.
- Do NOT build an employee phone app or any non-kiosk punch path now
  (a mobile PWA is future scope, only if needed).
- NEVER send face data to any outside AI service; store encrypted face
  templates, not raw photos. QR-punch snapshots are auto-deleted after the
  retention period set by admin.
- NEVER trust client time, kiosk location or client face-match result;
  the server verifies every punch.
- NEVER hard-delete attendance or audit records.
- NEVER commit secrets; use .env (ignored) + .env.example.
- NEVER start building a milestone before its spec and plan are approved.
- NEVER build future-scope items (section 1).
- Money uses exact decimal arithmetic, never floating point. Salary screens
  and exports are labelled "Estimate".
- Keep the app portable: config via environment variables, no vendor-only
  features, so free-tier hosting can later move to Docker.
```

```
- Do not change an approved API contract without updating the spec.
- Do not add a new dependency without stating its licence and reason.

## 6. What Needs to Be Done (scope)
- Org setup: plants, geofence on a map, kiosk device registration, shifts.
- Employees: permanent/temporary, contractors, Excel import, QR ID cards.
- Face enrolment with consent (done by HR); face + liveness punch at kiosk.
- QR issue/revoke/expiry; QR punch at kiosk with photo snapshot.
- Anti-proxy engine: kiosk device token, kiosk GPS inside geofence, plant
  network allow-list, face/QR validity, server time + cool-down, impossible
  travel; rejection reasons stored.
- Offline kiosk: queue punches when internet drops, sync later (signed).
- Attendance processing: check-in/out, late, early, overtime, hours worked.
- Dashboard (live headcount per plant), reports (Excel/PDF), roles, audit
log.
- LAST: payable salary estimate:
  - Monthly salary = Basic + DA + HRA + Travel + other allowances (fixed
    structure per employee, saved by HR, with effective-from date).
  - Per-day salary = monthly salary / salary days (26, 30 or actual:
setting).
  - Payable days = present + paid leave + paid holidays/weekly offs
(settings).
  - Earned = per-day x payable days; overtime pay = OT hours x OT rate.
  - Deductions (PF, ESIC, TDS, advances) are TYPED IN by the admin.
  - Net payable = earned + overtime - deductions.
  - Temporary workers: hours x hourly rate (+ overtime).
  - Output: monthly salary sheet on screen + Excel export.
  - Example: Rs 30,000 / 30 = Rs 1,000/day; 18 days = Rs 18,000; + 4 h OT x
    Rs 150 = Rs 600; - Rs 2,000 deductions = Rs 16,600.
  - Wait for the client's answers (proposal Q9, Q10) before building it.

## 7. Development Roadmap (milestones)
- [ ] M0 Setup: repo, plugins, MCP, subagents, tech stack + SRS approved, CI
- [ ] M1 Foundation: login/roles, plants, kiosk devices, employees, design
      system; deploy skeleton to free tier
- [ ] M2 QR attendance end-to-end at the kiosk (with snapshot)
- [ ] M3 Face enrolment + face/liveness attendance at the kiosk
- [ ] M4 Anti-proxy engine + kiosk geofence + offline kiosk sync
- [ ] M5 Attendance processing, dashboards, reports
- [ ] M6 Testing, security review, documentation, free-tier deployment
- [ ] M7 Payable salary estimate + Excel export (after client answers Q9/Q10)
- [ ] Later: Docker package for the company server
- [ ] Future (not now): full payroll and statutory features

## 8. Workflow (Spec-Driven, milestone by milestone)
1. spec-writer -> docs/specs/M<n>-<name>.md
```

```
2. Team reviews spec -> Approved
3. Plan Mode -> docs/plans/M<n>-<name>-plan.md
4. Team reviews plan -> Approved
5. Build on branch milestone/M<n>-<name>; tests + small commits
6. test-engineer, code-quality-reviewer, security-reviewer + manual testing
   -> PR -> teammate approval -> merge
- When unsure, ask. Record decisions in docs/decisions/.

## 9. Commands
- To be filled after the stack is approved (run, test, lint, migrate,
deploy).

## 10. Configuration Notes
- All timestamps stored in UTC, shown in IST (Asia/Kolkata).
- Face match threshold, kiosk GPS accuracy limit, punch cool-down, snapshot
  retention, salary-day basis, paid-day and overtime rules are settings.
- Hosting now: free tiers (frontend Vercel; backend + DB Render/Railway or
  similar). Camera needs HTTPS. Later: Docker on the company server.
- Design references: docs/design-refs/ (inspiration.md + screenshots).
  Sources: Godly, Dribbble, Awwwards, Mobbin, Figma Community, 21st.dev,
  Aceternity UI, Magic UI (free MIT components only).
```

9. SRS / specification document outline

This outline becomes `docs/SRS.md` (whole system) and, cut down, `docs/specs/_TEMPLATE.md` (one feature). It follows IEEE 830 / ISO 29148 structure, which examiners recognise.

1. **Introduction**
   a. Purpose of the document and intended readers (client, team, examiners)
   b. Scope: Upasthiti website (gate kiosk screen + admin dashboard) for all Sheet Shapers plants
   c. Definitions: punch, geofence, liveness, embedding, kiosk, regularisation, proxy attendance
   d. References: client meeting notes, DPDP Act 2023, Labour Codes, CLAUDE.md
2. **Problem statement**
   a. Current process and its failure (proxy punches from outside premises)
   b. Impact (payroll leakage, unreliable headcount, safety risk)
   c. Two worker types and why one method does not fit both
   d. Success metric: proxy punches reduced to zero; less than 5 seconds per punch at the gate

3. **Overall description**
    a. Product perspective and context diagram
    b. User classes and roles (Super Admin, Plant Admin, HR, Supervisor, plus the kiosk device identity) with permissions matrix
    c. Operating environment: free-tier cloud hosting (later the company server), kiosk tablets/phones/laptops at plant gates, admin browsers
    d. Design and implementation constraints (see section 7)
    e. Assumptions and dependencies (hardware supplied, plant network, SMTP available)
4. **Functional requirements** (FR-xx, each with priority and acceptance criteria)
    a. Organisation: plants, geofences, devices, shifts, holidays
    b. Employees and contractors: CRUD, import, ID card, lifecycle
    c. Face enrolment and consent
    d. QR issue, rotate, revoke
    e. Kiosk punch flow (face, QR, offline)
    f. Kiosk device registration and verification (device token, kiosk geofence, plant network)
    g. Anti-proxy engine rules and rejection reasons
    h. Attendance processing (pairing, late, OT, hours)
    i. Leave and regularisation workflows
    j. Dashboards, muster roll, analytics
    k. Reports and exports; salary structure and payable salary estimate (components, per-day rate, payable days, overtime, manual deductions, Excel export)
    l. Notifications, audit log, admin settings
5. **API contracts**
    a. Conventions: REST, `/api/v1`, JWT bearer, JSON error format, pagination, versioning
    b. Endpoint table: method, path, role, request schema, response schema, error codes
    c. Key flows: `POST /punch/kiosk`, `POST /kiosk/register`, `POST /enrol/face`, `POST /qr/issue`, `POST /qr/revoke`, `POST /kiosk/sync`, `GET /attendance/daily`, `GET /reports/muster`
    d. Auto-generated OpenAPI spec link
6. **Non-functional requirements**
    a. Performance: punch decision under 2 s on CPU; 300 punches in 10 minutes per gate
    b. Availability: kiosk works offline up to 72 h; system 99% during shifts
    c. Security: TLS, encryption at rest, RBAC, audit, OWASP Top 10 checks

   d. Privacy: DPDP compliance, retention, data-subject requests
   e. Accuracy: face false-accept rate target below 0.1%; false-reject below 3%
   f. Usability: 3 languages, large touch targets, works for low-literacy users
   g. Maintainability: Docker one-command install, pinned versions, backups, runbook
   h. Portability: runs on any Linux server with 4 cores, 8 GB RAM
7. **Constraints**
   a. No paid services or licence fees; permissive open-source only
   b. Free-tier cloud hosting first, Docker on the company server later; no paid runtime services
   c. Company-provided hardware; CPU-only face AI
   d. Indian legal compliance; academic timeline (final-year calendar)
8. **Edge cases and error handling**
   a. GPS spoofing, poor GPS accuracy indoors, geofence boundary jitter
   b. Photo/video/mask spoofing; twins; face changes (beard, helmet, safety glasses)
   c. QR screenshot shared, QR of a revoked or expired worker, damaged card
   d. Network down at kiosk; clock drift; duplicate punches; night shifts crossing midnight
   e. Employee transferred between plants; worker with no check-out
   f. Face match fails repeatedly: fallback path with supervisor approval
   g. Each error: code, user message (3 languages), log entry, admin alert rule
9. **Acceptance criteria**
   a. Given/When/Then for every FR
   b. End-to-end demo scenarios: temp worker QR at gate; permanent staff face at gate; proxy attempt from home rejected; offline kiosk sync; monthly register export; salary estimate for an employee with a fixed salary structure
10. **Data model**: ER diagram and table dictionary
11. **Milestones**: M0 to M6 with dates, owners and exit criteria (from CLAUDE.md roadmap)
12. **Appendices**: compliance mapping table, UI wireframes / Figma links, risk register, glossary

10. Skills, plugins, MCP servers, connectors and subagents

The starter kit (section 7) asks Claude Code to set most of these up by itself; you only approve its commands and do the one-time logins. Start small: install the first item in each group, add the rest when the milestone needs it. Names of marketplace plugins change,

so check `/plugin` for the current list. All of these are development-time tools; none ships inside Upasthiti, so none creates a licence cost for the client.

Claude Code plugins (from the official marketplace via `/plugin`)

- **feature-dev:** guided explore → design → build flow; fits the spec-driven workflow.
- **security-guidance:** warns on risky patterns while Claude edits; essential with biometrics.
- **pr-review-toolkit / code-review:** multi-angle PR review before a human approves.
- **commit-commands:** clean commits, push and PR creation in one command.
- **frontend-design:** pushes Claude toward distinctive, non-generic UI.

Skills

- **frontend-design** (Anthropic): design quality for dashboard, kiosk and PWA.
- **webapp-testing** (Anthropic, Playwright-based): lets Claude drive the running app and check flows.
- **xlsx / pdf / docx** (Anthropic): generate muster rolls, registers, the project report.
- **skill-creator:** build your own skills, e.g. `attendx-spec` (writes specs in your template) and `compliance-check` (checks a feature against the DPDP mapping).

MCP servers (inside Claude Code)

- **GitHub MCP:** issues, PRs, project board from Claude.
- **Playwright MCP:** browser automation for E2E testing and screenshots of UI.
- **Context7:** current, version-correct docs for FastAPI, React, OpenCV etc.; reduces outdated code.
- **Figma Dev Mode MCP:** turn M2's Figma frames into components with exact spacing and tokens.
- **shadcn MCP:** browse and add shadcn/ui and registry components.
- **21st.dev Magic MCP:** generate animated UI components; check each generated component's licence before keeping it.
- **PostgreSQL MCP (read-only, dev DB only):** inspect schema and data while debugging.

Claude.ai connectors (for planning, research and docs)

- **Figma, Canva:** design references and presentation slides.
- **Google Drive / Docs / Notion:** shared meeting notes, client requirements, report drafts.
- **Linear or Asana:** sprint board if you prefer it to GitHub Projects.
- **Gmail / Google Calendar:** client meeting scheduling and follow-ups.

- **Web search + Research:** compliance updates and new feature ideas.

Subagents (`.claude/agents/*.md`)

Subagent	Job	Tools / model hint
`spec-writer`	Writes a feature spec from `_TEMPLATE.md`, lists open questions, never writes code	Read, Write (docs only)
`feature-researcher`	Researches better approaches, libraries and their licences, new feature ideas	Read, WebSearch, WebFetch
`security-reviewer`	OWASP Top 10, auth/RBAC, secrets, biometric data handling, anti-proxy bypass attempts	Read, Grep, Bash (read-only)
`code-quality-reviewer`	Style, CLAUDE.md rule violations, duplication, complexity, naming, type safety	Read, Grep
`test-engineer`	Writes and runs pytest/Vitest/Playwright tests from acceptance criteria; reports coverage	Read, Write, Bash
`git-committer`	Stages related changes, writes Conventional Commit messages, opens PR with checklist	Bash (git, gh)
`compliance-auditor`	Checks a feature against the DPDP / Labour Code mapping in SRS appendix	Read
`ui-designer`	Proposes layout, motion and accessibility for a screen using design references	Read, WebFetch, Figma MCP

Create each with `/agents` → Create new agent, then commit the files so both seat holders share the same agents.

Design inspiration sources

Figma Community, Dribbble, Godly, Awwwards, Mobbin (real app flows), Refero, Land-book, 21st.dev, Aceternity UI, Magic UI, Motion Primitives, uiverse.io, LottieFiles. Use free, MIT-licensed components only and record the source in the PR.

11. Initial action steps

Do these in order; steps 1 to 4 happen before anyone opens Claude Code.

- [ ] 1. Get Sheet Shapers' answers to the Section 8 questions of the proposal (plants, worker counts, shifts, devices, hosting, salary rules Q9/Q10). Tell them we chose gate-kiosk-only attendance (answer to Q2).
- [ ] 2. Get written approval (WhatsApp or email) of the final scope: proposal v1.1, kiosk-only, salary estimate built last.
- [ ] 3. Install on the seat holders' laptops: Claude Code, Git, GitHub CLI (`gh`), Node.js LTS and Python 3.12. Run `gh auth login` once.
- [ ] 4. Apply for the free GitHub Student Developer Pack (gives GitHub Pro, needed for branch protection on private repos).
- [ ] 5. Unzip the Upasthiti starter kit into a new empty folder.
- [ ] 6. Open Claude Code in that folder and paste `KICKOFF_PROMPT.md` (Part A, normal mode). Approve its commands; do the logins it asks for.
- [ ] 7. When Part A is done, switch to Plan Mode and type `Start Part B`. Answer its questions and approve the plan.
- [ ] 8. Review the SRS and subagents it creates with the whole team.
- [ ] 9. Add teammates to the GitHub repo; pick a design direction from `docs/design-refs/inspiration.md`.
- [ ] 10. Start milestone M0, then follow the six-gate flow for every milestone.